

ESRG

EMBEDDED
SYSTEMS
RESEARCH GROUP

University of Minho
School of Engineering

MSc in Industrial Electronics Engineering and Computers
Embedded Systems

Pipeline Report

Project Report

Project Group
Class of 25-26

Project supervised by
Professor Adriano Tavares, PhD

Contents

1	Introduction	2
2	System Architecture	2
2.1	Top-level structure	2
2.2	Address model	2
3	Pipeline Organization	2
3.1	Stage partition	2
3.2	Writeback stage trade-offs	3
3.3	Register boundaries	3
3.4	Register file visibility	4
3.5	Stage Behavior	4
3.5.1	IF stage	4
3.5.2	ID stage	5
3.5.3	EX stage	5
3.5.4	MEM stage	6
3.6	Interstage Register Behavior	6
3.6.1	IF/ID register	7
3.6.2	ID/EX register	7
3.6.3	EX/MEM register	8
4	Hazard Resolution	9
4.1	Hazard map	9
4.2	Structural hazards	10
4.3	Forwarding and ordinary RAW hazards	10
4.4	Load-use interlock	10
4.5	Memory handshake stalls	11
4.6	Control hazards	11
4.7	Condition-code and carry dependencies	12
5	Interrupt Semantics	12
6	Validation	12
7	Load-Use Interlock and AGI Pipelines	13
8	Conclusion	13

1 Introduction

This report documents the implemented 4-stage in-order pipelined RISC processor and its SoC-level integration. It builds on prior work adapting Gray’s gr0040 single-cycle design and describes the transition to a stage-based pipeline organization.

The discussion focuses on architecturally visible behavior: system structure and addressing, stage responsibilities, hazard handling, interrupt semantics, and validation outcomes. The final sections relate the implementation to classical load-use interlock terminology and summarize the key implications for control and latency.

The active core is the stage-based CPU in `srcs/m_cpu.v`. It instantiates IF, IF/ID, ID, ID/EX, EX, EX/MEM, and MEM stages together with hazard and forwarding logic. The resulting machine is a four-stage in-order pipeline with three explicit interstage registers.

2 System Architecture

2.1 Top-level structure

The top-level `soc` module in `srcs/m_soc.v` instantiates the CPU, instruction ROM, data RAM, and peripheral bus. Instruction fetch and data access are physically separated: ROM services the instruction path and RAM services the data path. This organization removes the single-memory structural hazard of a unified-memory design. [3]

2.2 Address model

The ISA uses 16-bit instructions and 16 general-purpose registers. Instruction addresses are byte addresses; sequential execution advances the PC by 2. Both ROM and RAM are indexed by `addr[9:1]`, so the memories are word-addressed internally even though the architectural interface is byte-addressed.

The EX stage in `srcs/m_ex_stage.v` exports the data address as `o_d_ad = (_sum << 1)`, so address arithmetic is performed in word units and converted to byte addresses at the stage output.

3 Pipeline Organization

The implemented pipeline is a four-stage in-order design with separate instruction and data memories, MEM-integrated writeback, EX/MEM operand forwarding, explicit load-use interlocking, ID-stage control-flow resolution, and interrupt entry integrated into the main control path. The source tree exposes that organization directly: `srcs/m_cpu.v` instantiates IF, IF/ID, ID, ID/EX, EX, EX/MEM, and MEM blocks, then wires the hazard, forwarding, and register-file paths around them.

3.1 Stage partition

The implemented stage order is IF, ID, EX, MEM. In `srcs/m_cpu.v`, the CPU derives `_wb_rf_we`, `_wb_wa`, and `_wb_wd` directly from MEM-stage outputs, so a standalone writeback (WB) stage is not present in the active core.

Stage	Implemented function
IF	Presents the current PC to the instruction side, aligns PC with the returned instruction, and suppresses post-redirect fetch outputs during the drain window.
ID	Decodes the instruction, reads the register file, constructs immediates, resolves BX/JAL control flow, emits hazard metadata, and decides whether the instruction actually enters EX.
EX	Selects operands, applies EX/MEM forwarding, performs ALU and address computation, forms store data, and produces candidate status values and pre-writeback data.
MEM	Drives the external load/store interface, keeps the pipeline frozen while <code>i_rdy</code> is low, and supplies the final register-file writeback value for EX-issued instructions.

Table 1: Implemented stage responsibilities.

3.2 Writeback stage trade-offs

A separate WB stage and MEM-integrated writeback both have valid use cases. Table 2 summarizes the practical trade-offs for this design.

Option	Pros	Cons
Separate WB stage	Clearer stage boundaries and a dedicated commit point; can ease MEM-stage fan-out and naturally support a distinct MEM/WB forwarding source.	Adds one pipeline stage and associated control/state; can increase redirect recovery distance and verification surface.
Writeback in MEM	Shorter pipeline depth and lower structural overhead; keeps ALU/load result commit close to data-return handling.	Concentrates memory-handshake and architectural register commit responsibilities in MEM; leaves no distinct MEM/WB forwarding boundary.

Table 2: Trade-offs between adding a standalone WB stage and keeping writeback in MEM.

This project chooses MEM-integrated writeback. The shorter depth is therefore achieved by collapsing WB into MEM.

3.3 Register boundaries

The active core does not use one uniform commit stage for every instruction class. Decode-only instructions take effect when the ID stage fires, while EX-issued instructions become architecturally visible through the MEM-stage writeback path.

Table 3: Architectural effect boundaries in the active core.

Family	Representative instructions	Practical commit boundary
Decode-only control	IMM, CLI, STI, BX, IRET	Commits in ID, gated by <code>_id_fire</code> .
EX-issued data/ALU/memory	RR, RI, ADDI, LW/LB, SW/SB, GETCC, SETCC	Dispatches through ID/EX and completes in MEM; any register write is sourced from MEM.

Family	Representative instructions	Practical commit boundary
Split-path control+link	JAL	Redirect decision commits in ID, while link-register writeback is produced in EX and written in MEM.

That split matters for documentation: calling MEM the “commit stage” is accurate only for instructions that traverse EX. Control-flow and interrupt-control instructions already change machine state at the ID boundary.

3.4 Register file visibility

The register file in `srcs/m_regfile16x16.v` is write-first on read/write collisions:

```
1 assign o_ra = (i_we && (i_wa != 4'b0000) && (i_ra == i_wa)) ? i_wd : _mem[i_ra];
2 assign o_rb = (i_we && (i_wa != 4'b0000) && (i_rb == i_wa)) ? i_wd : _mem[i_rb];
```

In practice, that makes MEM-stage register writes visible to ID in the same cycle and removes the need for a separate MEM/WB forwarding path. [2] Writes to `r0` are ignored (explicit zero register), so hazard detection treats `r0` as a non-dependency destination.

3.5 Stage Behavior

3.5.1 IF stage

In the IF stage, the processor fetches the instruction associated with the current PC and keeps that instruction aligned with the PC that launched it. The active design follows that model in `srcs/m_if_stage.v` by keeping a one-entry PC delay line (`_pc_d1`) and by qualifying `o_valid` with both the instruction-side hit signal and the post-flush drain counter. In the current `srcs/m_soc.v` integration, `i_hit` is deasserted only while reset is active, so outside reset the instruction side behaves like a fixed-latency source.

This stage also enforces redirect-drain semantics. Asserting `i_stall` holds IF state (`_pc_d1`, `o_pc`, and `_flush_cnt`) and, through top-level stall gating in `srcs/m_cpu.v`, prevents PC advancement. Asserting `i_flush` synchronously seeds `_pc_d1` and `o_pc` with `i_flush_pc` and sets `_flush_cnt` to `2'd2`; therefore `o_valid` is deasserted for the next two hit-qualified IF updates. If `i_hit` deasserts, bubble retirement pauses, so those two invalid outputs are deferred until hits resume rather than skipped.

```
1 assign o_valid = i_hit & (_flush_cnt == 2'd0);
2 assign o_insn = (_flush_cnt == 2'd0) ? i_insn : `CPU_NOP_INSN;
3
4 always @(posedge i_clk) begin
5     if (i_flush) begin
6         _pc_d1 <= i_flush_pc;
7         o_pc    <= i_flush_pc;
8         _flush_cnt <= 2'd2;
9     end else if (i_hit & ~i_stall) begin
10        _pc_d1 <= i_pc;
11        o_pc    <= _pc_d1;
12        if (_flush_cnt != 2'd0) begin
13            _flush_cnt <= _flush_cnt - 2'd1;
14        end
15    end
16 end
```

Listing 1: Redirect-drain behavior in `srcs/m_if_stage.v`

3.5.2 ID stage

In the ID stage, the fetched instruction is translated into register indices, immediates, hazard metadata and control signals. This is also where control flow is resolved in the active core, and as a result the branch decision is made earlier than it would be in an EX-resolved pipeline. The implementation in `srcs/m_id_stage.v` combines the control decoder, branch-decision unit, immediate builder, and register-file read data. The core does not implement a dynamic branch predictor (no BHT/BTB/RAS path): next-PC selection falls through to PC+2 by default, and redirection occurs only when ID produces a taken branch/jump that commits (`o_branch_take` gated by `_id_fire` in `srcs/m_cpu.v`). As such, we have adopted a static not-taken policy, with early resolution at the ID-stage. This stage also decides whether an instruction actually enters EX by driving `o_exec_valid`. Instructions such as IMM, CLI, STI, and BX are handled entirely in ID and never occupy ID/EX. JAL is split: its redirect target is resolved in ID, but its link value still travels through EX and MEM, which is required to make the link register writeback visible to the instruction's EX-stage.

This stage is where most control-side architectural effects become visible. When `_id_fire` is true in `srcs/m_cpu.v`, IMM updates the prefix latch, CLI/STI update `_gie`, and IRET raises `o_iret_detected`. The ID stage also emits `o_reads_rd` and `o_reads_rs` for the hazard unit, which is why load-use detection is expressed as a relationship between ID and ID/EX. Because the register file is write-first, the decode stage can also see a same-cycle MEM-stage write without a separate MEM/WB forwarding path.

```

1 assign _imm16 = i_imm_pre_state ? {i_i12_pre_state, _imm}
2           : {_sxi11, _i_4, _imm[3:1], _i_0};
3
4 assign o_branch_take   = i_valid & (_is_jal | (_is_bx & _bdu_take));
5 assign o_branch_target = _is_jal ? (_imm16 + i_rs_data) : (i_pc + _sxd16);
6
7 assign o_exec_valid = i_valid & ~(_is_imm | _is_cli | _is_sti | _is_bx);

```

Listing 2: Immediate, redirect, and EX-dispatch decisions in `srcs/m_id_stage.v`

3.5.3 EX stage

In the EX stage, the processor consumes the staged signals from the ID/EX register: it selects the correct operands, performs ALU operations, and emits the data needed by the later MEM stage. Forwarding happens in this stage because EX is where RAW hazards become concrete operand conflicts. In `srcs/m_ex_stage.v`, the stage receives the full ID/EX bundle and one explicit forwarding source from EX/MEM. That forwarding path is wired to `_exmem_wb_pre_data`, so it can carry ALU, JAL, and GETCC style results, but not load data. The same stage computes the data-side byte address as $(_sum \ll 1)$, emits store data from the forwarded Rd value, and produces both pre-writeback data and candidate status outputs.

The load case is the important exception. A load result does not exist until MEM, so EX/MEM forwarding cannot satisfy a true load-use dependency and the hazard unit must inject a bubble. EX is also the source of the status information observed by ID. The top-level CPU does not keep a clocked PSW register between EX and ID; instead, `_ccz`, `_ccn`, `_ccc`, `_ccv`, and `_c` are recomputed combinatorially from EX outputs in `srcs/m_cpu.v`. Despite not being a staged output, this simple solution removes the explicit CC/carry interlock.

```

1 assign _rd_fwd = (i_forward_a == 1'b1) ? i_exmem_wb_data : i_rd_data;
2 assign _rs_fwd = (i_forward_b == 1'b1) ? i_exmem_wb_data : i_rs_data;
3
4 assign o_d_ad      = (_sum << 1);
5 assign o_store_data = _rd_fwd;
6 assign o_wb_pre_data = i_is_getcc ? {11'b0, _psw_vector} : _alu_res;

```

Listing 3: Forwarding and data/result selection in `srcs/m_ex_stage.v`

3.5.4 MEM stage

In the MEM stage, the processor presents the EX result to the external data interface, waits for completion when necessary, and produces the final value used for register-file writeback. The implementation in `srcs/m_mem_stage.v` is a thin handshake wrapper around the EX/MEM bundle: it asserts the appropriate data-side strobes whenever the instruction is valid, raises `o_mem_wait` until `i_rdy` arrives for memory operations, and chooses between load data and the EX-generated pre-writeback value. The top-level CPU writes the register file only when `_mem_valid & _mem_rf_we` is true.

For EX-issued instructions, this is the only register-write retirement boundary. Non-memory instructions pass through MEM without waiting, whereas loads hold the upstream pipeline through the `o_mem_wait`, which waits for the memory to assert `i_rdy` before allowing the instruction to complete.

```

1 assign _is_mem_op = i_lw | i_lb | i_sw | i_sb;
2 assign o_mem_wait = i_valid & _is_mem_op & ~i_rdy;
3
4 assign o_valid    = i_valid & (~_is_mem_op | i_rdy);
5 assign o_rf_we    = i_rf_we;
6 assign o_wb_data = i_is_load ? i_data_in : i_wb_pre_data;

```

Listing 4: MEM-stage handshake and writeback selection in `srcs/m_mem_stage.v`

```

1 always @(posedge i_clk) begin
2     if (i_rst) begin
3         _loaded <= 1'b0;
4     end else if (_insn_ce) begin
5         _loaded <= 1'b0;
6     end else begin
7         _loaded <= (_lw | _lb);
8     end
9 end
10
11 assign _mem_rdy = ~((_lw | _lb) & ~_loaded);

```

Listing 5: Load-ready generation in `srcs/m_soc.v`

3.6 Interstage Register Behavior

The active core uses three explicit interstage registers. Each one is a conventional rising-edge storage element, but the *meaning* of stall and flush differs across the boundaries because control resolution happens in ID and writeback is collapsed into MEM.

Register	Payload	Stall behavior	Flush behavior
IF/ID	valid + pc + insn	Frozen by <code>_stall_id</code> .	Cleared on branch commit or IRQ accept.
ID/EX	Full execute bundle from ID	Frozen only by <code>_stall_ex</code> during <code>mem_wait</code> .	Cleared on IRQ accept and on load-use bubble injection.
EX/MEM	Memory and writeback bundle from EX	Frozen with ID/EX during <code>mem_wait</code> .	Cleared on IRQ accept only.

Table 4: Practical contract of the three interstage registers.

3.6.1 IF/ID register

The IF/ID register holds the fetch bundle stable so the decode stage sees one instruction at a time even when the front end is stalled. In `srcs/m_pipe_if_id.v`, this boundary carries only `valid`, `pc`, and `insn`; no decoded control crosses it. Reset and flush both clear `o_valid` and write a NOP into `o_insn`, while stalls simply preserve the previous contents.

In practice, this is the youngest explicit pipeline latch, so it is the first structure that must be cleared on a redirect. Branches and accepted interrupts both flush IF/ID because both invalidate the fetched instruction that was sitting behind the redirecting ID-stage instruction.

```

1  always @(posedge i_clk) begin
2      if (i_rst || i_flush) begin
3          o_valid <= 1'b0;
4          o_pc    <= 16'h0000;
5          o_insn  <= `CPU_NOP_INSN;
6      end else if (!i_stall) begin
7          o_valid <= i_valid;
8          o_pc    <= i_pc;
9          o_insn  <= i_insn;
10     end
11 end

```

Listing 6: IF/ID register behavior in `srcs/m_pipe_if_id.v`

3.6.2 ID/EX register

The ID/EX register snapshots the execute bundle: operands, immediate, destination register, and the control bits that EX needs in the following cycle. In `srcs/m_pipe_id_ex.v`, it is the widest register in the design because the full decoded execute bundle crosses this boundary. The important integration detail lives in `srcs/m_cpu.v`: `i_valid` is driven by `_id_exec_valid & _id_fire`, so decode-only instructions never enter ID/EX at all.

This same boundary is also where the design injects a load-use bubble. During a pure load-use hazard, IF and ID freeze while ID/EX is cleared to zero, creating a one-cycle bubble in EX without disturbing EX/MEM. During `mem_wait`, by contrast, the register is not cleared; it is frozen so the same instruction remains pending in EX while EX/MEM is held stable.

```

1  pipe_id_ex u_pipe_id_ex (
2      .i_stall(_stall_ex),
3      .i_flush(_flush_idex),
4      .i_valid(_id_exec_valid & _id_fire),
5      ...
6  );

```


Listing 7: ID/EX control wiring in `srcs/m_cpu.v`

```

1  always @(posedge i_clk) begin
2      if (i_rst || i_flush) begin
3          o_valid      <= 1'b0;
4          o_rf_we      <= 1'b0;
5          o_lw         <= 1'b0;
6          o_lb         <= 1'b0;
7          o_sw         <= 1'b0;
8          o_sb         <= 1'b0;
9          o_is_load    <= 1'b0;
10     end else if (!i_stall) begin
11         o_valid      <= i_valid;
12         o_rf_we      <= i_rf_we;
13         o_lw         <= i_lw;
14         o_lb         <= i_lb;
15         o_sw         <= i_sw;
16         o_sb         <= i_sb;
17         o_is_load    <= i_is_load;
18     end
19 end

```

Listing 8: ID/EX storage behavior in `srcs/m_pipe_id_ex.v`

3.6.3 EX/MEM register

The EX/MEM register preserves the side-effecting outputs of EX long enough for the memory system and writeback path to observe them reliably. In `srcs/m_pipe_ex_mem.v`, it carries the memory strobes, data address, store payload, pre-writeback value, destination register, and load classification. It is stalled together with ID/EX on `mem_wait`, which keeps the same instruction visible to MEM until the external transaction completes.

The branch case is the important distinction at this boundary. When a branch resolves in ID, the older instruction already in EX is still on the correct path and is allowed to complete normally, so EX/MEM must *not* be cleared. On IRQ acceptance, however, the design squashes the younger in-flight work and redirects the architectural PC save path to `r14`, so EX/MEM is explicitly flushed.

```

1  pipe_ex_mem u_pipe_ex_mem (
2      .i_stall(_stall_ex),
3      .i_flush(_accept_irq),
4      .i_valid(_ex_valid),
5      ...
6  );

```

Listing 9: EX/MEM control wiring in `srcs/m_cpu.v`

```

1  always @(posedge i_clk) begin
2      if (i_rst || i_flush) begin
3          o_valid      <= 1'b0;
4          o_rf_we      <= 1'b0;
5          o_lw         <= 1'b0;
6          o_lb         <= 1'b0;

```

```

7      o_sw          <= 1'b0;
8      o_sb          <= 1'b0;
9      o_is_load     <= 1'b0;
10     end else if (!i_stall) begin
11         o_valid     <= i_valid;
12         o_rf_we     <= i_rf_we;
13         o_lw        <= i_lw;
14         o_lb        <= i_lb;
15         o_sw        <= i_sw;
16         o_sb        <= i_sb;
17         o_is_load   <= i_is_load;
18     end
19 end

```

Listing 10: EX/MEM storage behavior in `srcs/m_pipe_ex_mem.v`

4 Hazard Resolution

Hazards in the active core are best understood by asking where the dependence becomes visible in the four-stage pipeline. Structural contention between instruction fetch and data access has already been removed by the top-level Harvard organization. The remaining cases are data availability hazards between stages, variable data-side latency in MEM, redirect hazards caused by control flow, and the special case of condition-code visibility.

4.1 Hazard map

Table 5: Hazard classes in the active four-stage pipeline.

Class	Where it arises	Implemented handling
Structural	IF and MEM would conflict in a unified-memory design.	Removed at the SoC level by separate instruction ROM and data RAM, so fetch and data access do not contend.
RAW (non-load)	Older producer in EX/MEM, younger consumer entering EX from ID/EX.	Resolved by EX/MEM-to-EX forwarding in <code>srcs/m_forwarding.v</code> and <code>srcs/m_ex_stage.v</code> ; no stall is inserted.
Load-use	Older load in ID/EX, younger consumer still in ID.	Detected by <code>srcs/m_hazard_unit.v</code> ; IF and ID stall and ID/EX is cleared to create one bubble in EX.
Memory wait	An EX-issued load/store reaches MEM while <code>i_rdy</code> is low.	<code>srcs/m_mem_stage.v</code> asserts <code>o_mem_wait</code> ; the hazard unit freezes the upstream pipeline until the transaction completes.
Control redirect	BX/JAL resolves in ID or an interrupt is accepted.	Younger work is flushed, and the IF stage suppresses two later valid outputs through its redirect drain counter.
CC/carry visibility	ID needs flags/carry produced by the current EX stage.	No explicit stall is inserted; <code>srcs/m_cpu.v</code> exposes EX-generated status combinationally back to ID.

4.2 Structural hazards

The active design does not suffer from the classic unified-memory structural hazard. At the SoC level, `srcs/m_soc.v` feeds instruction fetch from ROM and data accesses from RAM or MMIO, so IF and MEM do not compete for one physical memory port. That architectural choice removes one whole hazard class before the pipeline-control logic is even involved. [3]

4.3 Forwarding and ordinary RAW hazards

Most register RAW hazards appear when an older producer has already reached EX/MEM and a younger consumer is just entering EX through ID/EX. In that situation, waiting for the MEM-stage writeback path would be unnecessary and would introduce avoidable stalls. The active core resolves that case with one explicit forwarding path from EX/MEM into EX.

The forwarding unit in `srcs/m_forwarding.v` compares the source-register identifiers of the instruction currently in EX against the destination register of the valid instruction in EX/MEM. If there is a nonzero register match and EX/MEM intends to write that destination, `srcs/m_ex_stage.v` selects the forwarded value instead of the register-file read value. This is why ordinary ALU-to-ALU and ALU-to-store-address dependencies do not stall.

```

1 assign o_ForwardAE = (i_rdE != 4'h0) && (i_rdE == i_WriteRegM) &&
2                   i_RegWriteM && i_validM;
3 assign o_ForwardBE = (i_rsE != 4'h0) && (i_rsE == i_WriteRegM) &&
4                   i_RegWriteM && i_validM;
```

Listing 11: EX/MEM forwarding in `srcs/m_forwarding.v`

This implementation provides only one explicit forwarding source: EX/MEM to EX. It does not implement a distinct MEM/WB forwarding path because the active core has no standalone WB stage. Same-cycle MEM-to-ID visibility is instead provided by the write-first register file. [2]

4.4 Load-use interlock

The remaining true data hazard is the load-use case. It occurs one stage earlier than the forwarding case: the older instruction is a load currently sitting in ID/EX, while the younger instruction is still in ID and is trying to read the register that the load will eventually produce. At that point the load data does not yet exist, so forwarding cannot solve the dependence.

The hazard unit in `srcs/m_hazard_unit.v` detects exactly that situation by comparing the register-read intent of the instruction in ID against the destination register of the load in ID/EX. The dependence is ignored for `r0`, because writes to `r0` are architecturally discarded, as per the ABI.

```

1 assign _load_use_hazard = i_id_valid & i_idex_valid & i_idex_is_load &
2                   (i_idex_rd != 4'h0) & _match_idx;
3
4 assign o_stall_if      = i_mem_wait | _decode_hazard;
5 assign o_stall_id      = i_mem_wait | _decode_hazard;
6 assign o_stall_ex      = i_mem_wait;
7 assign o_flush_idxex  = _accept_irq || (_decode_hazard & ~i_mem_wait);
```

Listing 12: Load-use detection and control in `srcs/m_hazard_unit.v`

When a true load-use hazard is detected and MEM is not already waiting, IF and ID stall, ID/EX is flushed to zero, and EX/MEM is allowed to keep advancing. In pipeline terms, the younger instruction stays parked in ID while one bubble is injected into EX. The same policy

also covers store-data dependencies, because stores assert `o_reads_rd` in the decoder. A load immediately followed by a store of the loaded register therefore takes the same interlock path as an ALU consumer.

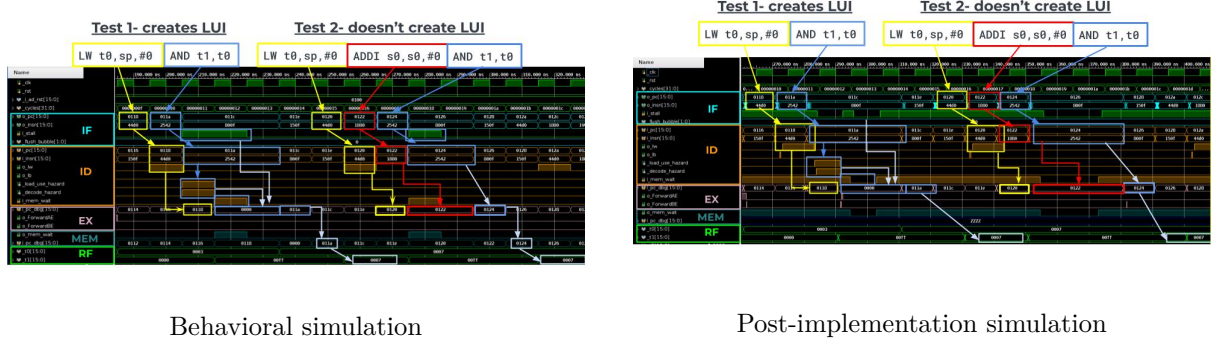


Figure 1: Waveforms for a true load-use pair and for a one-instruction-gap case.

4.5 Memory handshake stalls

The MEM stage introduces a different kind of hazard: not a producer-consumer dependency, but variable completion latency on the data side. This hazard arises when an EX-issued load or store reaches MEM and the external data interface is not yet ready to complete the operation. In that case the pipeline must stop younger work from overtaking the in-flight memory access.

The implementation in `srcs/m_mem_stage.v` expresses that condition as `o_mem_wait`. The hazard unit turns that signal into a full upstream freeze: IF stops advancing the PC, IF/ID holds its contents, and both ID/EX and EX/MEM are frozen through `o_stall_ex`. Unlike the load-use case, no bubble is inserted here, because the instruction already occupying EX/MEM must remain visible to MEM until the transaction finishes.

At the SoC level, the current BRAM integration makes that behavior observable for loads. In `srcs/m_soc.v`, `_mem_rdy` is delayed by one cycle for LW/LB, so a core load consumes one address cycle and one completion cycle on the data side. Stores use the same handshake wires but, in the current SoC integration, do not incur that extra ready delay.

4.6 Control hazards

Control hazards arise when the sequential fetch stream turns out to be wrong. In this design, that happens in two ways: a BX or JAL redirects the PC from the ID stage, or an interrupt is accepted and vectors control to `i_irq_vector`. Because branch resolution occurs in ID rather than EX, the younger wrong-path work is concentrated at the front of the pipeline rather than several stages later.

The hazard unit therefore flushes the youngest affected state rather than indiscriminately clearing the whole machine. IF/ID is flushed on any committed branch redirect or accepted interrupt. ID/EX is flushed on interrupt acceptance and on load-use bubble insertion, but not on an ordinary taken branch. That distinction is important: when a branch resolves in ID, the older instruction already in EX is still on the correct path and must be allowed to complete normally.

The IF stage adds one more practical effect on top of the register flushes. On a redirect, `srcs/m_if_stage.v` loads `_flush_bubble <= 2'd2`; since `o_valid` is asserted only when that counter is zero, the IF stage suppresses two later valid outputs while the redirect drains through the fetch alignment logic. In other words, the visible control-hazard cost is not just the IF/ID flush itself, but also the two invalid IF-valid opportunities that follow the redirect if the instruction side continues to hit.

4.7 Condition-code and carry dependencies

The active pipeline does not stall on condition-code or carry dependencies. Instead of maintaining a separate clocked PSW stage and interlocking on it, `srcs/m_cpu.v` derives `_ccz`, `_ccn`, `_ccc`, `_ccv`, and `_c` combinationaly from the current EX-stage outputs `_ex_new_*` and `_ex_carry_we`. The ID stage therefore observes EX-generated status information in the same cycle through direct combinational feedback.

This behavior is explicit in the RTL. The design avoids a dedicated CC/carry interlock, but it also means that status visibility is defined by this specific combinational path rather than by a staged PSW register.

5 Interrupt Semantics

Interrupt entry is integrated into the pipeline control path. The hazard unit accepts an interrupt only when `_accept_irq = i_irq_take & ~i_mem_wait`.

When `_accept_irq` is asserted, the CPU:

- writes PC-2 into `r14`;
- redirects the PC to `i_irq_vector`;
- clears the global interrupt enable latch;
- clears any in-flight IMM prefix state.

The CPU also edge-latches the IRQ request. A level-sensitive interrupt therefore produces one acceptance pulse per assertion interval, not one acceptance pulse per cycle.

Interrupt acceptance has explicit priority over branch commit. The CPU defines `_id_fire = _ifid_valid & ~_stall_id & ~_accept_irq` and `_branch_take_commit = _id_branch_take & _id_fire`. If a branch is present in ID in the same cycle as interrupt acceptance, the branch does not commit.

6 Validation

This report includes only testbenches that compile against the active stage-based core. Each bench listed below was rebuilt and rerun against the current source set. Table 6 summarizes the coverage relevant to the pipeline control path.

Table 6: Validation benches used by this report.

Bench	Property confirmed
<code>tb_pipe_branch_flush</code>	Taken branch flushes younger work and preserves the architected result of the branch path.
<code>tb_pipe_branch_memwait_defer</code>	A branch present in ID does not commit while MEM is stalled.
<code>tb_pipe_mem_wait_freeze</code>	<code>mem_wait</code> freezes the PC and the upstream pipeline state.
<code>tb_pipe_r0_load_no_hazard</code>	A load targeting <code>r0</code> does not create a false dependency stall.
<code>tb_pipe_imm_prefix_flush</code> , <code>tb_pipe_imm_prefix_irq_clear</code>	Prefix state is cleared correctly by redirects and by interrupt entry.
<code>tb_pipe_irq_branch_priority</code>	Interrupt acceptance suppresses a simultaneous branch commit.
<code>tb_pipe_irq_mem_wait_deferral</code>	Interrupt acceptance is deferred until <code>mem_wait</code> drops.
<code>tb_pipe_irq_oneshot_level</code>	Level-sensitive IRQ input is accepted once per assertion interval.
<code>tb_pipe_irq_precise_boundary</code>	Interrupt entry saves <code>r14</code> and asserts in-IRQ state at the precise pipeline boundary.

Bench	Property confirmed
<code>tb_pipe_fuzz_invariants</code>	No X-propagation on PC or data address and no illegal <code>accept_irq</code> & <code>mem_wait</code> combination in bounded randomized execution.

7 Load-Use Interlock and AGI Pipelines

Golden and Mudge define the *load-use interlock* (LUI) pipeline as the classical five-stage RISC organization in which load hazards are handled by an interlock and address-generation hazards do not occur. They define the *address-generation interlock* (AGI) pipeline as a rearranged organization that removes the load-use hazard by moving address generation later, at the cost of address-generation interlocks and later branch resolution. [5]

Table 7: Trade-offs between LUI-style and AGI-style organizations for this core.

Style	Pros	Trade-offs
LUI-style (implemented)	Keeps address generation in EX and uses an explicit load-use interlock; fits the current ID-stage control-flow resolution and verified hazard/forwarding contracts.	True load-use pairs still incur an interlock bubble; load-use latency is managed rather than structurally removed.
AGI-style	Can remove the classical load-use hazard by moving address generation later in the pipeline.	Introduces address-generation interlocks, tends to move branch effects later, and requires a substantial repartitioning of this core’s stage and control contracts.

The pipeline implemented in this repository is not an AGI pipeline. Address generation still occurs in EX, the load-use interlock is explicit in the hazard unit, and the memory interface still occupies a later stage.

This project therefore adopts the LUI-style organization because it matches the existing RTL partition (address generation in EX, ID-stage control-flow resolution, and explicit hazard interlocks) and preserves the validated control behavior.

The active design differs from the classical LUI pipeline in one specific way: it collapses WB into MEM. The reduction in stage count comes from removing the standalone writeback stage, not from moving address generation after cache access. [5]

8 Conclusion

The implemented processor is a four-stage in-order pipeline with separate instruction and data memories, MEM-integrated writeback, EX/MEM operand forwarding, explicit load-use interlocking, ID-stage branch resolution, and precise interrupt entry.

The implementation details that determine latency and hazard behavior are:

- one explicit forwarding source, EX/MEM to EX;
- write-first register-file visibility from MEM to ID in the same cycle;

- one decode bubble for a true load-use hazard plus one data-side ready cycle for BRAM loads;
- two suppressed IF-valid opportunities after a taken redirect;
- combinational EX-to-ID condition-code feedback in place of a registered PSW interlock.

In the terminology of Golden and Mudge, this processor remains a load-use interlock design. Its shorter depth is obtained by eliminating the explicit WB stage while preserving address generation in EX and maintaining interlocked handling of true load-use dependencies. [5]

References

- [1] A. P. Shanthi, “Simple Operation of MIPS,” University of Maryland. <https://www.cs.umd.edu/users/meesh/411/SimpleMips.htm>.
- [2] C. Kozyrakis, “Pipeline Hazards,” EE108B Lecture 9, Stanford University. <https://web.stanford.edu/class/archive/ee/ee108b/ee108b.1082/handouts/lect.09.PipelineHazards.pdf>.
- [3] Cornell CS3410 course notes, “Hazards I,” Cornell University. <https://www.cs.cornell.edu/courses/cs3410/2014sp/lecture/09-hazards-i-g-notes.pdf>.
- [4] University of Maryland course material, “Dealing with Data Hazards,” <https://www.cs.umd.edu/~meesh/411/mips-pipe/proj-fall11/mips-doc/node7.html>.
- [5] M. Golden and T. Mudge, “Comparison of two common pipeline structures,” *IEE Proceedings – Computers and Digital Techniques*, vol. 143, no. 3, pp. 161–167, 1996.